I. Dokumentacja do lab. nr 5 - "Składowe statyczne i modyfikator final"

II. Roch Majkowski - grupa 1, semestr III

III. Przedmiot - "Programowanie obiektowe"

IV. Opis zadania do realizacji

Do zrealizowania były następujące zadania:

1. Napisz klasę `Product` z polami instancyjnymi `name`, `price`, `category`, `available` oraz metodą `printInfo()`.
2. Dodaj do klasy `Product` statyczne pole `productCount`, które zwiększa się przy tworzeniu każdego obiektu.
3. Utwórz klasę `LibraryMember` z polem `final memberId` i metodą wypisującą dane czytelnika.
4. Napisz klasę `VisitCounter`, która pokazuje różnicę między zmienną lokalną a polem klasy podczas wielokrotnego wywołania metody.
5. Zaimplementuj klasę `ScoreBoard`, gdzie pola instancyjne opisują wynik jednego meczu, a pole statyczne liczy wszystkie rozegrane mecze.
6. Przygotuj krótki eksperyment: utwórz kilka obiektów tej samej klasy i pokaż, że każdy ma własny stan, ale współdzielą pola statyczne.
7. Napisz klasę `ImmutableColor` oznaczoną jako `final`, z polami `final r`, `g`, `b` inicjalizowanymi w konstruktorze oraz metodą `final describe()`. Spróbuj (w komentarzu) utworzyć podklasę tej klasy oraz nadpisać metodę `describe` i zapisz, jaki błąd kompilacji otrzymujesz.

V. Technologie wykorzystane w zadaniu

- Java

VI. Realizacja zadania

1. Zadanie - klasa `Product`

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `Product`

```java
package Product;

public class Product {
    String name;
    double price;
    String category;
    boolean available;

    public Product(String name, double price, String category, boolean available) {
        this.name = name;
        this.price = price;
        this.category = category;
        this.available = available;
    }

    public void printInfo() {
        System.out.println("Name: " + name +
                           "\nPrice: " + price +
                           "\nCategory: " + category +
                           "\nAvailable: " + available + "\n");
    }
}
```

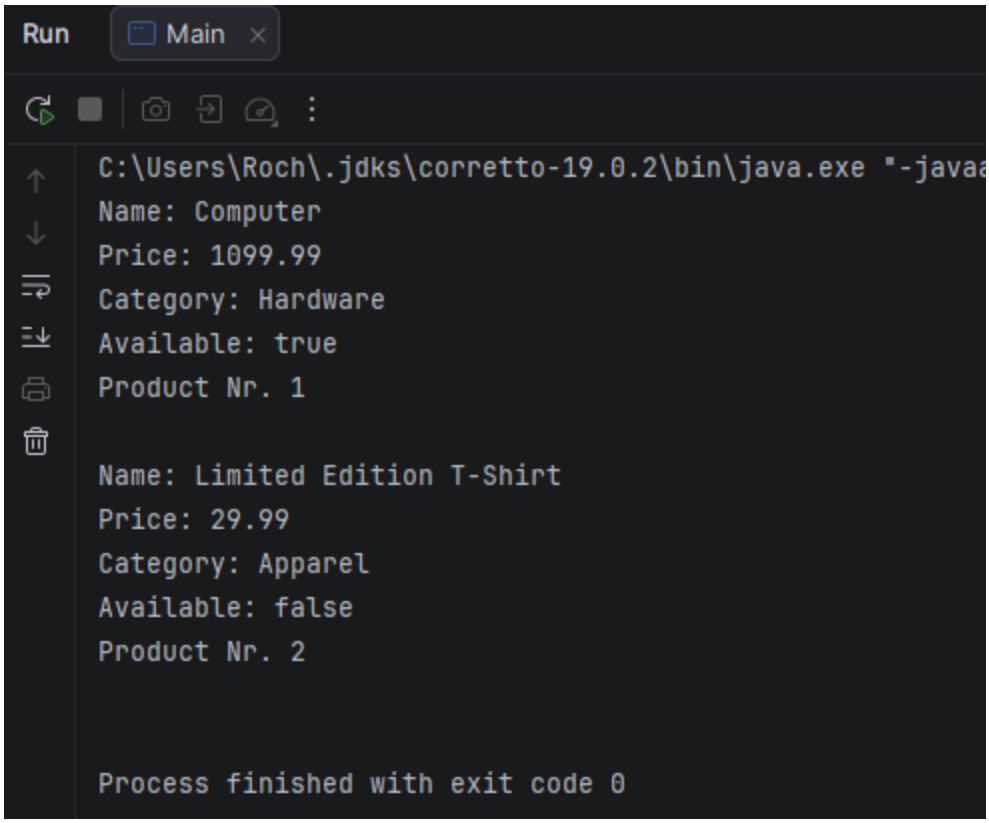
Klasa `Main`

```java
package Product;

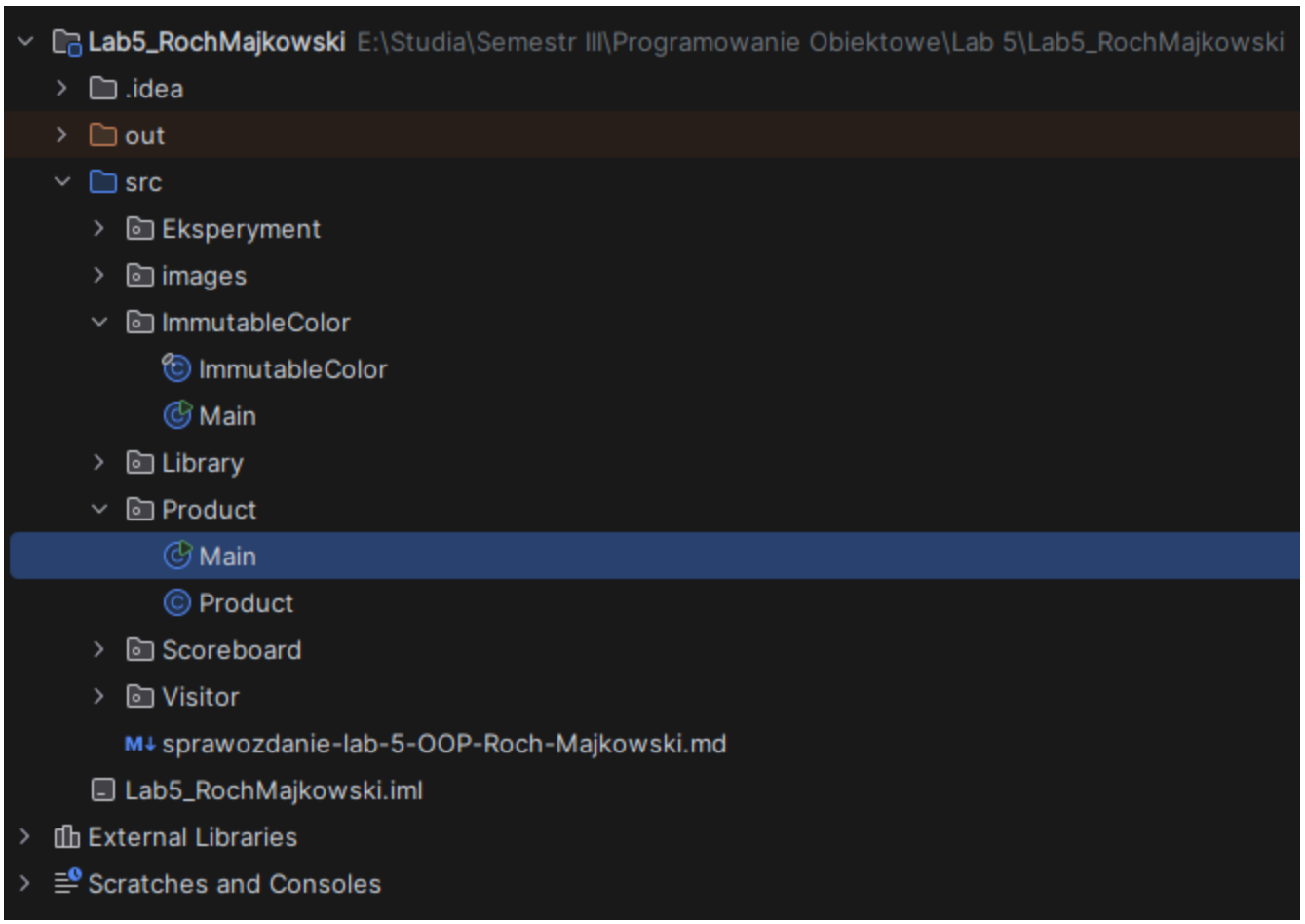
public class Main {
    public static void main(String[] args) {
        Product product1 = new Product("Computer", 1099.99, "Hardware", true);
        Product product2 = new Product("Limited Edition T-Shirt", 29.99, "Apparel", false);

        product1.printInfo();
        product2.printInfo();
    }
}
```

1a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



1b. Struktura projektu/programu:



1c. Wnioski:

- Pola `name`, `price`, `category`, `available` są instancyjne, więc każdy obiekt `Product` przechowuje własne wartości.
- Konstruktor ustawia pola przy tworzeniu obiektu, a metoda `printInfo()` wypisuje dane konkretnego produktu.
- Dwa utworzone produkty (`product1`, `product2`) mają różne dane i nie wpływają na siebie nawzajem.

2. Zadanie - statyczne pole `productCount`

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `Product`

```
package Product;

public class Product {
    String name;
    double price;
    String category;
    boolean available;

    static int productCount = 0;

    public Product(String name, double price, String category, boolean available) {
        this.name = name;
        this.price = price;
        this.category = category;
        this.available = available;

        Product.productCount++; // Zwiększenie licznika przy tworzeniu każdego obiektu
    }

    public void printInfo() {
        System.out.println("Name: " + name +
                           "\nPrice: " + price +
                           "\nCategory: " + category +
                           "\nAvailable: " + available + "\n");
    }
}
```

Klasa `Main`

```
package Product;

public class Main {
    public static void main(String[] args) {
        System.out.println("Liczba produktów na starcie: " + Product.productCount);

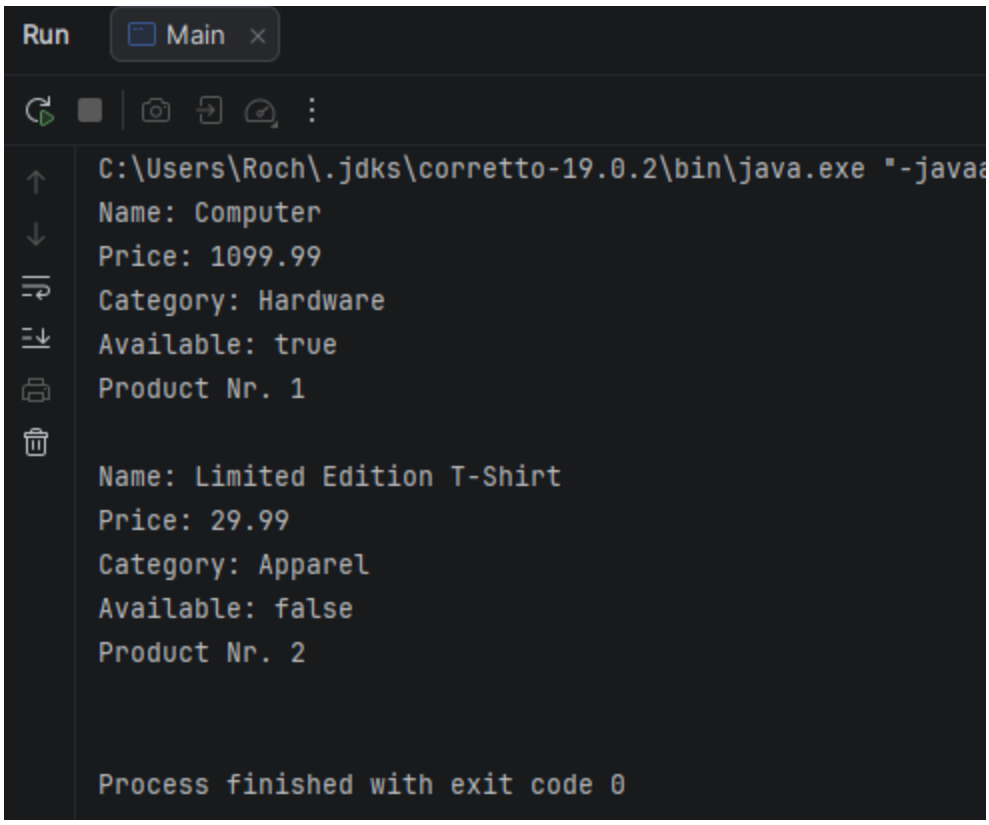
        Product product1 = new Product("Computer", 1099.99, "Hardware", true);
        System.out.println("Po utworzeniu product1: " + Product.productCount);

        Product product2 = new Product("Limited Edition T-Shirt", 29.99, "Apparel", false);
```

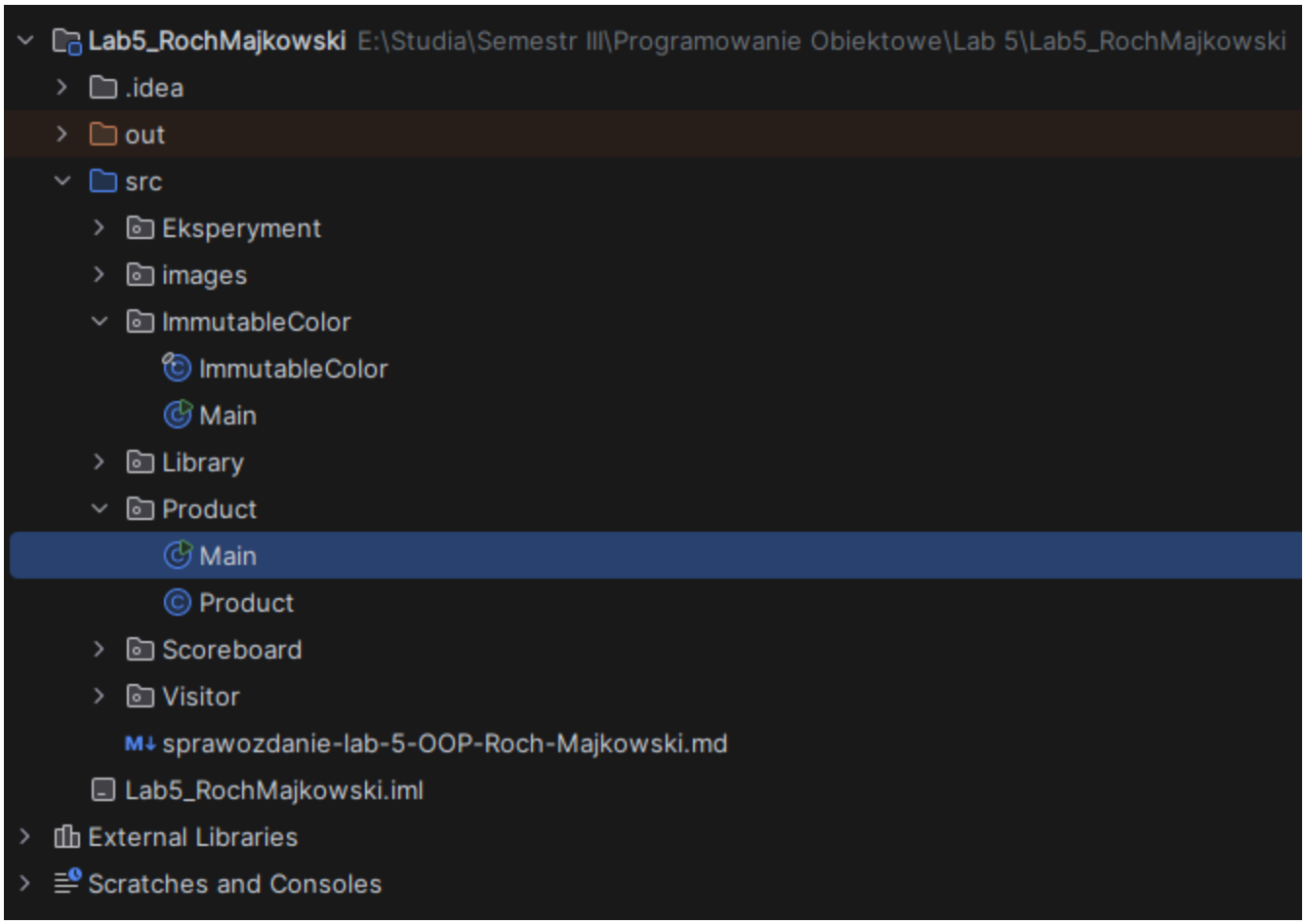
```
        System.out.println("Po utworzeniu product2: " + Product.productCount + "\n");

        product1.printInfo();
        product2.printInfo();
    }
}
```

2a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



2b. Struktura projektu/programu:



2c. Wnioski:

- Pole `productCount` jest `static`, więc istnieje jedno dla całej klasy, a nie osobno dla każdego obiektu.
- Licznik zwiększany jest w konstruktorze (`Product.productCount++`), dlatego rośnie przy tworzeniu każdego obiektu: 0 na starcie, 1 po `product1`, 2 po `product2`.
- Do pola statycznego można odwołać się przez nazwę klasy (`Product.productCount`), bez tworzenia obiektu.

3. Zadanie - klasa `LibraryMember` z polem `final`

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `LibraryMember`

```
package Library;

public class LibraryMember {
    final int memberId;
    String name;
    boolean isRenting;

    public LibraryMember(int memberId, String name, boolean isRenting){
        this.memberId = memberId;
```

```
        this.name = name;
        this.isRenting = isRenting;
    }

    public void memberInfo() {
        System.out.println(
                            "\nId: " + memberId +
                            "\nName: " + name +
                            "\nRenting: " + isRenting
        );
    }
}
```

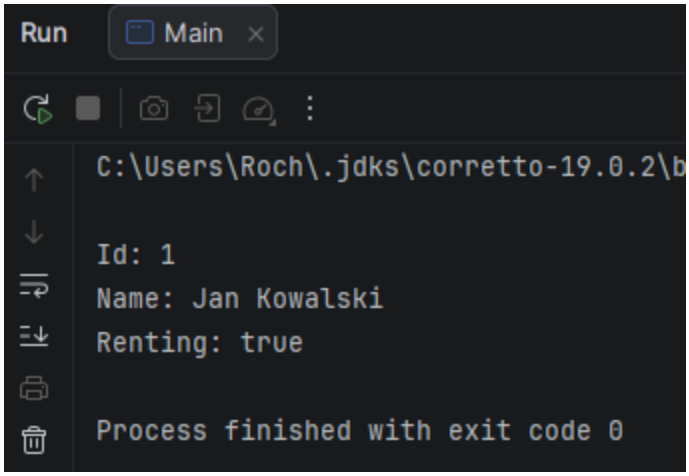
Klasa `Main`

```
package Library;

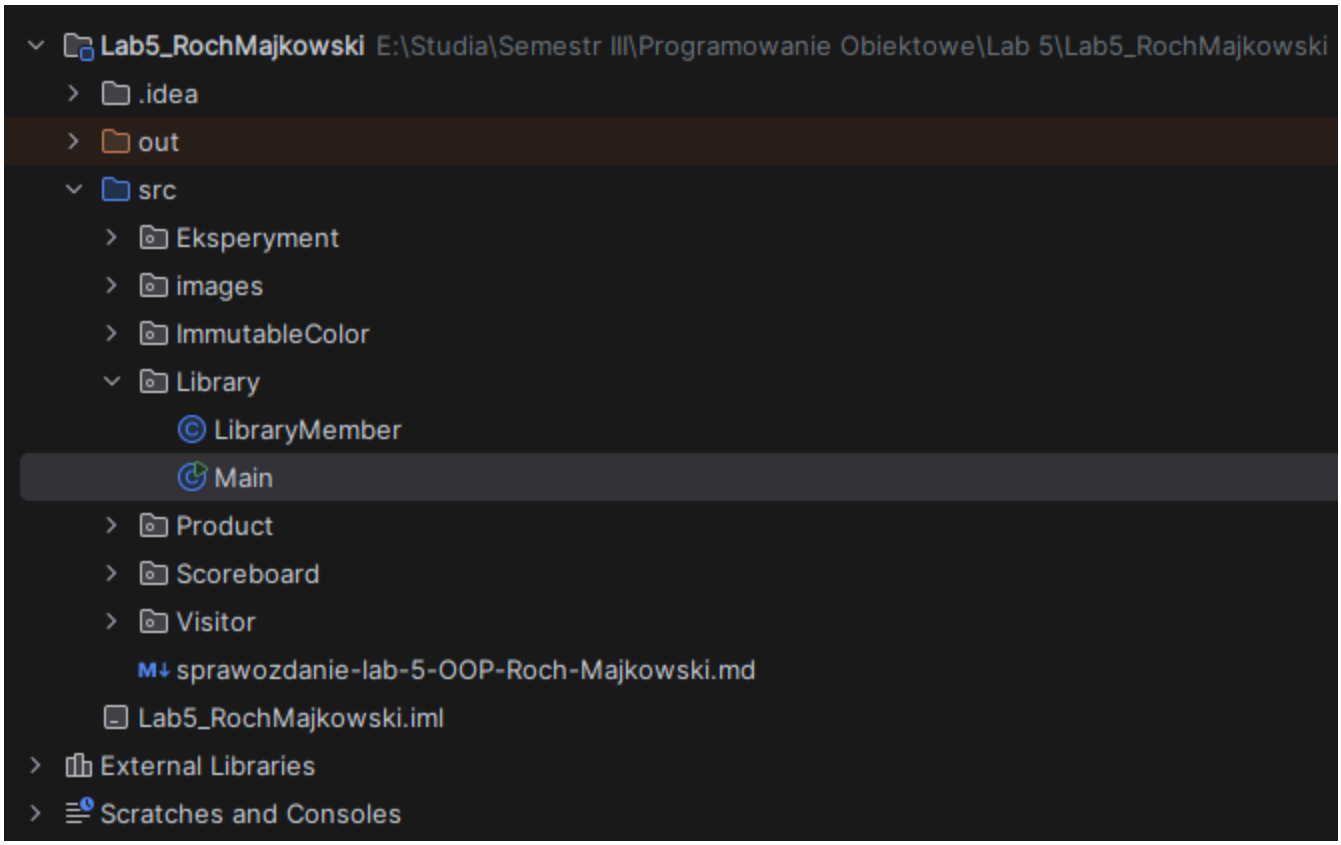
public class Main {
    public static void main(String[] args) {
        LibraryMember libraryMember = new LibraryMember(1, "Jan Kowalski", true);
        libraryMember.memberInfo();
        // libraryMember.memberId = 2; // Błąd : java: cannot assign a value to final variable memberId

    }
}
```

3a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



3b. Struktura projektu/programu:



3c. Wnioski:

- Pole `final int memberId` można ustawić tylko raz.
- Metoda `memberInfo()` wypisuje dane czytelnika: id, imię, nazwisko oraz, czy wypożycza.
- Próba zmiany pola po utworzeniu obiektu (`libraryMember.memberId = 2;`) kończy się błędem kompilacji: `java: cannot assign a value to final variable memberId` .
- Pola `name` i `isRenting` nie są `final` , więc można je zmieniać.

4. Zadanie - klasa `VisitCounter` - zmienna lokalna a pole klasy

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `VisitCounter`

```
package Visitor;

public class VisitCounter {
    int visited;

    public void counter() {

        for (int x=0; x<5; x++) {
```

```
            int localVisited = 0;
            localVisited++; // Inkrementacja lokalnej
            visited++; // Inkrementacja globalnej

            System.out.println("\nWizyta lokalna: " + localVisited +
                    "\nWizyta globalna: " + visited);
        }

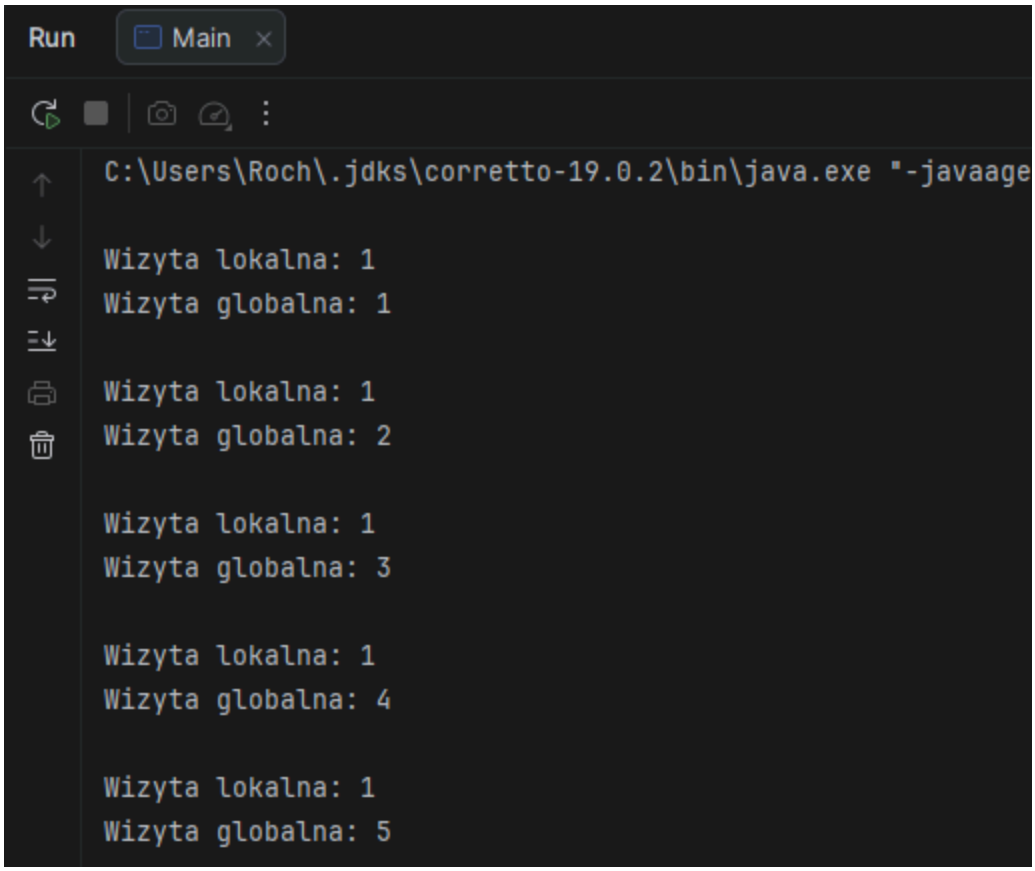
    }
}
```

Klasa `Main`

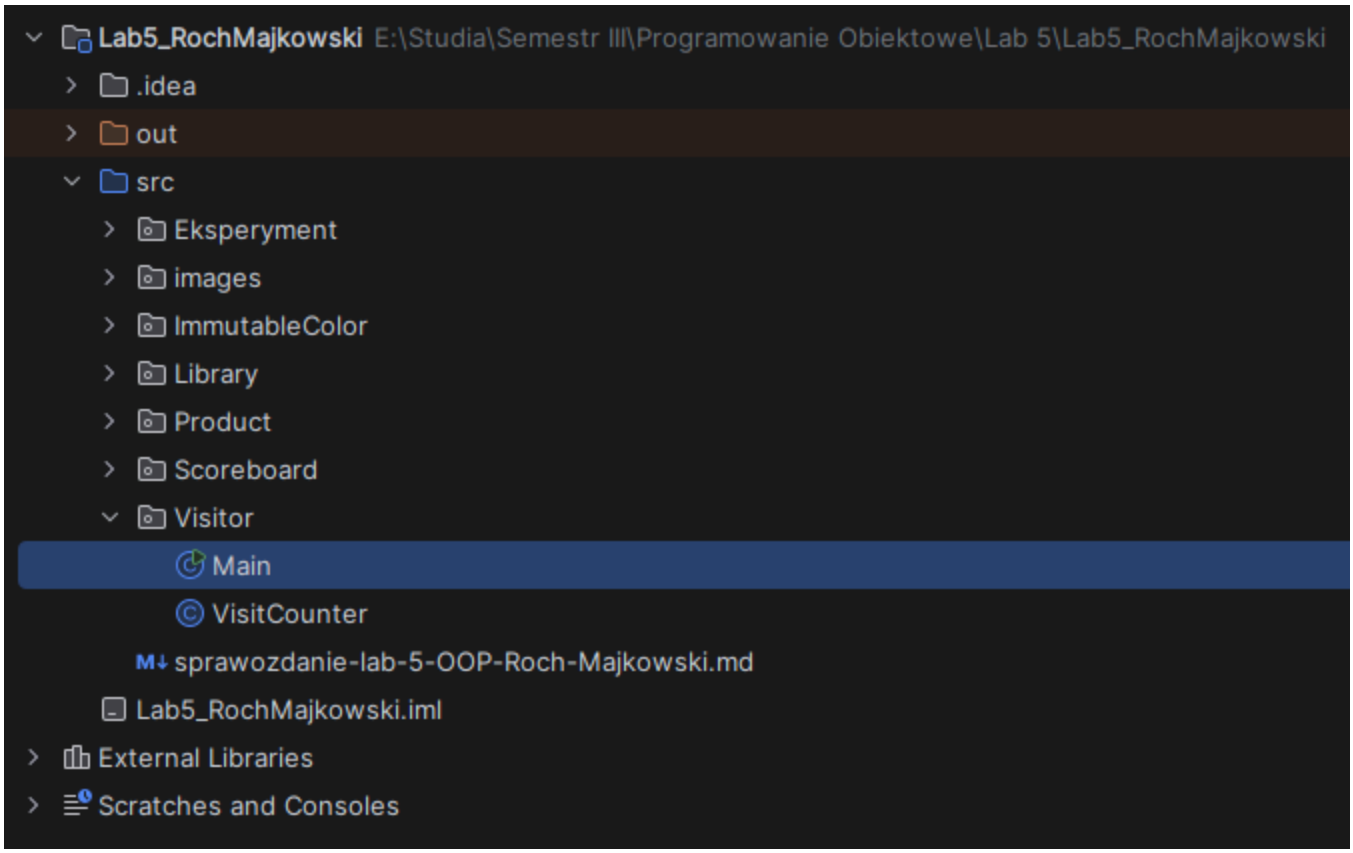
```
package Visitor;

public class Main {
    public static void main(String[] args) {
        VisitCounter visitCounter = new VisitCounter();
        visitCounter.counter();
    }
}
```

4a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



4b. Struktura projektu/programu:



4c. Wnioski:

- Zmienna lokalna `localVisited` jest tworzona od nowa w pętli, dlatego zawsze wynosi 1.
- Pole `visited` rośnie od 1 do 5.
- Zmienna lokalna istnieje tylko w bloku, w którym ją zadeklarowano, pole przez cały czas życiu obiektu.

5. Zadanie - klasa `ScoreBoard`

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `ScoreBoard`

```
package Scoreboard;

public class ScoreBoard {
    String teamA;
    String teamB;
    int teamA_score;
    int teamB_score;
    static int playedMatches;

    public ScoreBoard(String teamA, String teamB, int teamA_score, int teamB_score) {
        this.teamA = teamA;
        this.teamB = teamB;
        this.teamA_score = teamA_score;
        this.teamB_score = teamB_score;
```

```
    }

    public void printMatchHistory() {

        ScoreBoard.playedMatches++; // Zwiększenie numeru meczy o 1

        if (teamA_score > teamB_score){
            System.out.println("\nTeam "+ teamA + " has won the game against "+ teamB +", with a score of "+tea
                    "Match Nr." + playedMatches + ", "+ teamA +" vs. "+ teamB + "\n");
        }

        else if (teamA_score < teamB_score){
            System.out.println("\nTeam "+ teamB + " has won the game against "+ teamA +", with a score of "+tea
                    "Match Nr." + playedMatches + ", "+ teamA +" vs. "+ teamB + "\n");
        }

        else {
            System.out.println("\nIt's a tie!\n" +
                    "Match Nr." + playedMatches + ", "+ teamA +" vs. "+ teamB + "\n");
        }

    }


}
```

Klasa `Main`

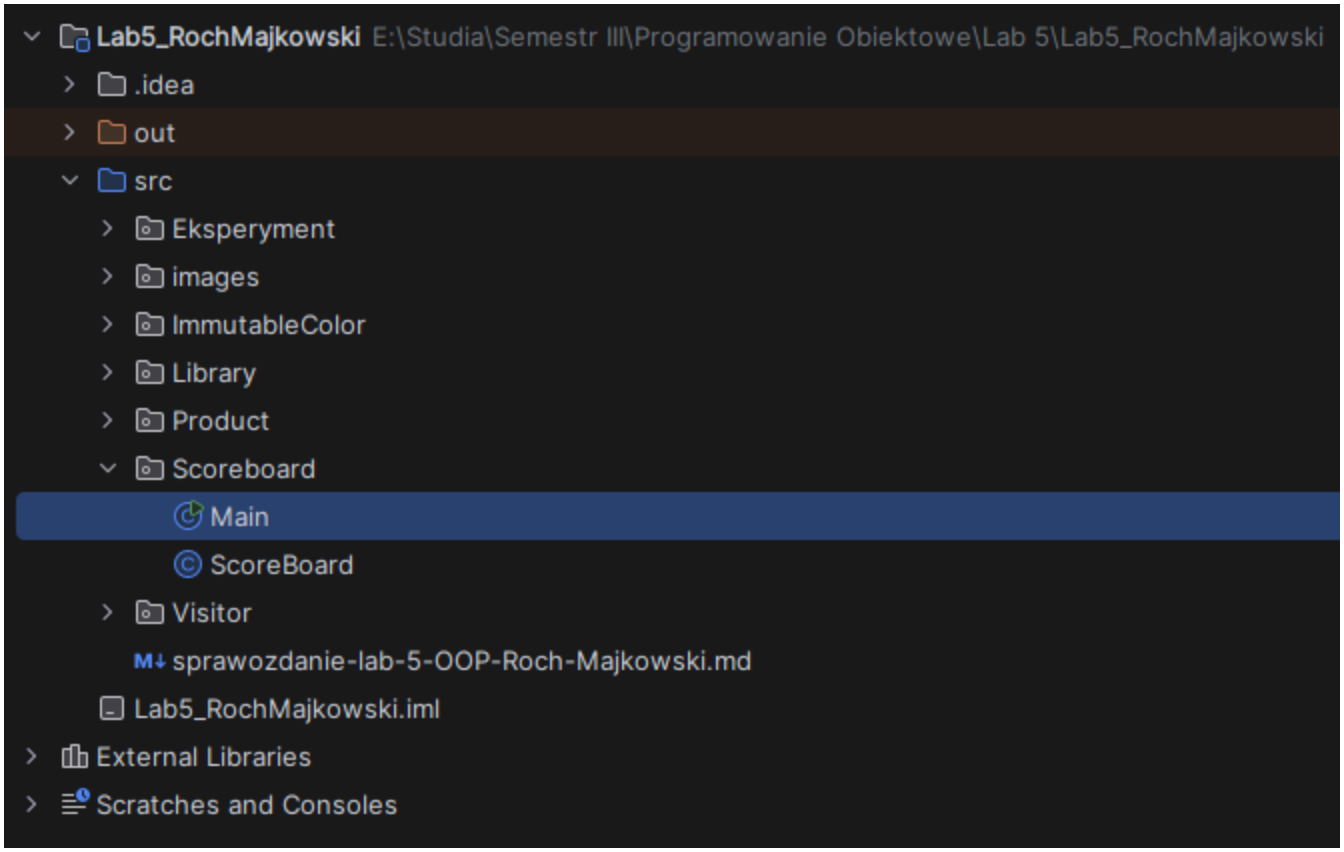
```
package Scoreboard;

public class Main {
    public static void main(String[] args) {
        ScoreBoard scoreBoard = new ScoreBoard("Blekitni", "Gryf", 3, 5);
        ScoreBoard scoreBoard1 = new ScoreBoard("Strikers", "Keepers", 8, 1);
        ScoreBoard scoreBoard2 = new ScoreBoard("Ying", "Yang", 2, 2);
        scoreBoard.printMatchHistory();
        scoreBoard1.printMatchHistory();
        scoreBoard2.printMatchHistory();
    }
}
```

5a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



5b. Struktura projektu/programu:



5c. Wnioski:

- Pola instancyjne (`teamA` , `teamB` , `teamA_score` , `teamB_score`) opisują wynik jednego meczu.
- Pole statyczne `playedMatches` liczy wszystkie rozegrane mecze.
- Metoda porównuje wyniki i wypisuje zwycięzcę albo remis.

6. Zadanie - eksperyment ze stanem obiektów

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `Counter`

```
package Eksperyment;

public class Counter {
    String name;
    int value;

    static int obiekty = 0;

    public Counter(String name, int value){
        this.name = name;
        this.value = value;
        Counter.obiekty++;
    }

    public void printInfo(){
        System.out.println(name + " -> wartość: " + value + ", ilość obiektów: " + obiekty);
    }
}
```

Klasa `Main`

```
package Eksperyment;

public class Main {
    public static void main(String[] args) {
        Counter c1 = new Counter("c1", 10);
        Counter c2 = new Counter("c2", 20);
        Counter c3 = new Counter("c3", 30);

        // Przed zmianą wartości - wywołanie

        c1.printInfo();
        c2.printInfo();
        c3.printInfo();

        // Zmiana dla c1

        c1. value = 256;

        // Po zmianie c1 = 256;

        c1.printInfo();
        c2.printInfo();
        c3.printInfo();

        // Zmiana pola liczenia obiektow

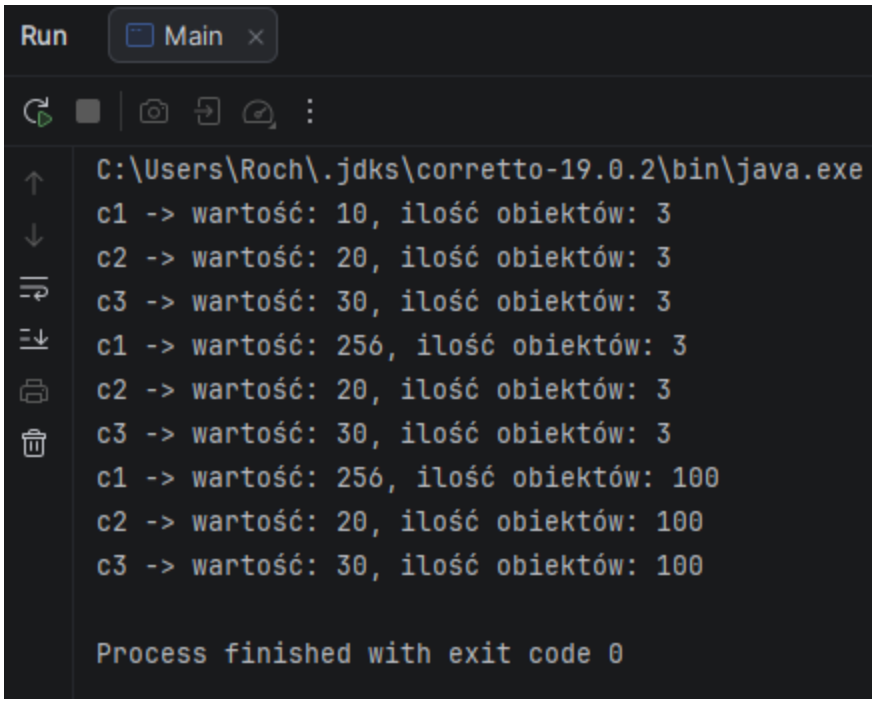
        Counter.obiekty = 100;
```

```
        //Po zmainie wszystkich obiektow jest 100

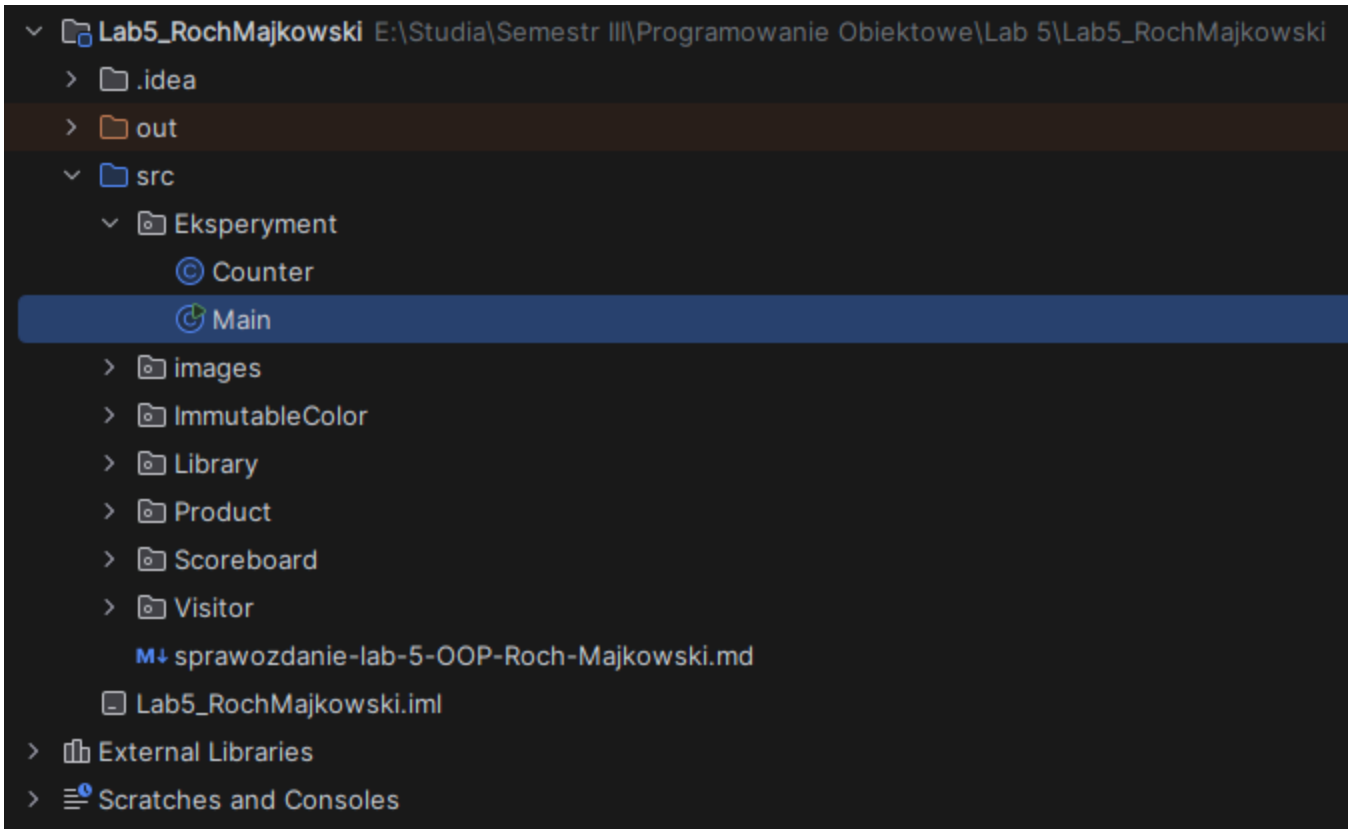
        c1.printInfo();
        c2.printInfo();
        c3.printInfo();


    }
}
```

6a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



6b. Struktura projektu/programu:



6c. Wnioski:

Utworzono trzy obiekty klasy `Counter` (`c1`, `c2`, `c3`). Zmiana pola instancyjnego `value` w obiekcie `c1` (z 10 na 256) nie wpłynęła na `c2` ani `c3`, ponieważ każdy obiekt ma własną kopię tego pola. Natomiast zmiana pola statycznego `obiekty` (z 3 na 100) była widoczna we wszystkich trzech obiektach, bo pole statyczne należy do klasy i jest współdzielone przez wszystkie jej instancje.

Wynik programu:

- przed zmianą: `c1`, `c2`, `c3` mają wartości 10, 20, 30, a `obiekty` = 3,
- po `c1.value = 256`: zmienia się tylko `c1` (256), pozostałe bez zmian,
- po `Counter.obiekty = 100`: wszystkie trzy obiekty pokazują `obiekty` = 100.

7. Zadanie - klasa `ImmutableColor` z `final` metodą i `final` klasą

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `ImmutableColor`

```
package ImmutableColor;

public final class ImmutableColor {
    final int r;
```

```
    final int g;
    final int b;

    public ImmutableColor(int r, int g, int b) {
        this.r = r;
        this.g = g;
        this.b = b;
    }

    public final void describe() {
        System.out.println("RGB (" + r + ", " + g + ", " + b + ")");
    }
}
```

Klasa `Main`

```
package ImmutableColor;

public class Main {
    public static void main(String[] args) {
        ImmutableColor red = new ImmutableColor(255, 0, 0);
        red.describe();

        // Próba utworzenia podklasy klasy ImmutableColor i nadpisania metody describe():



        /*
        class MyColor extends ImmutableColor {
            public MyColor(int r, int g, int b) {
                super(r,g,b);
            }

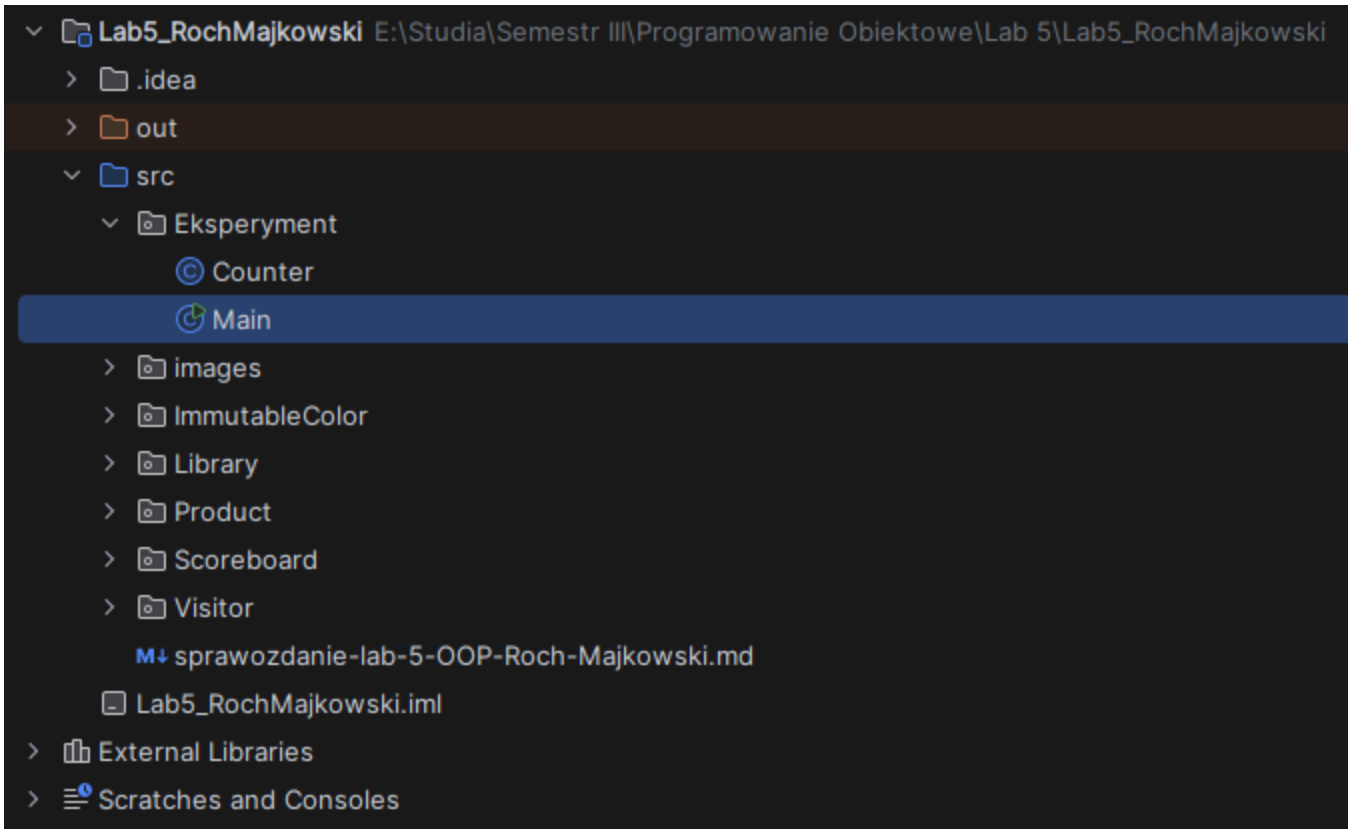
            @Override
            public void describe() {
                System.out.println("Nowy opis");
            }
        }
        */



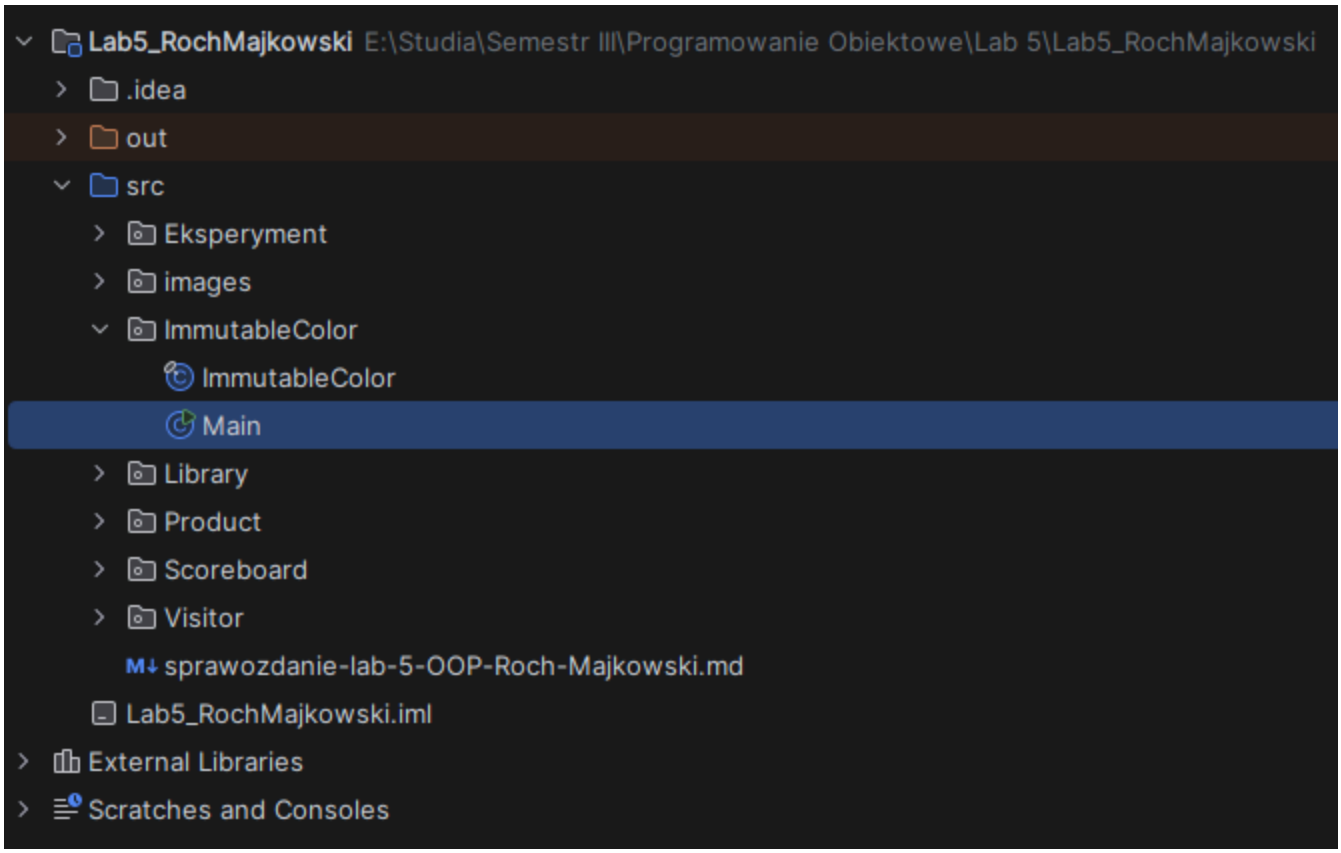
        //Błąd kompilacji : java: cannot inherit from final ImmutableColor.ImmutableColor

    }
}
```

7a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



7b. Struktura projektu/programu:



7c. Wnioski:

- Próba utworzenia podklasy `MyColor extends ImmutableColor` daje błąd kompilacji: `java: cannot inherit from final ImmutableColor.ImmutableColor` .